

IS2203: Object Oriented Programming

Abstraction



Viraj Welgama

Motivation

- Let's assume that you are driving a Car...
 - You use accelerator, brake, steering wheel
 - You **don't need to know** how the engine or gearbox works internally
- **Abstraction** = Hiding unnecessary details & showing only the essentials

Definition of Abstraction

- Abstraction is the process of:
 - Hiding implementation details
 - Showing only essential features
- Focuses on **what an object does**, not how it does it
- Helps in managing complexity

Why Abstraction is Important?

- Reduces **complexity** in large systems
- Enhances code **readability** and **maintainability**
- Supports **reusability** of code
- Provides **security** by exposing only necessary information

Abstraction vs Encapsulation

Aspect	Abstraction	Encapsulation
Focus	Hides implementation details	Bundles data + methods together
Implementation	Achieved via abstract classes/interfaces	Achieved via access modifiers (private, public, etc.)
Example	"What a car does"	"How car's data is hidden"

Example in Real Applications

- Banking System
 - User sees deposit, withdraw, check balance
 - Internal implementation (DB queries, authentication) is hidden
- ATM Machine
 - User interacts via simple buttons/menus
 - Complex backend logic is abstracted

C++ Implementation

```
#include <iostream>
using namespace std;

// Abstract class
class Shape {
public:
    // Pure virtual function (no implementation)
    virtual void draw() = 0;

    // Concrete method
    void info() {
        cout << "This is a shape" << endl;
    }
};

// Derived class 1
class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing a Circle" << endl;
    }
};

// Derived class 2
class Rectangle : public Shape {
public:
    void draw() override {
        cout << "Drawing a Rectangle" << endl;
    }
};
```